**Name:Manjyot Wadhwa**
**Roll no :65**
**Div:D15C**

# Experiment 1

# Implementation of Linear and Logistic Regression

## 1. Dataset Source

Dataset Name: **Heart Disease Dataset**
Source: Kaggle
Link: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

The dataset was downloaded from Kaggle and used in CSV format.

---

## 2. Dataset Description

The dataset contains medical attributes used to determine the presence of heart disease.

### 📊 Dataset Size

- 303 samples
- 14 attributes

### 📌 Features Include:

- age
- sex
- cp (chest pain type)
- trestbps (resting blood pressure)
- chol (cholesterol)
- fbs (fasting blood sugar)
- restecg (resting ECG results)
- thalach (maximum heart rate)
- exang (exercise induced angina)
- oldpeak
- slope
- ca (number of major vessels)
- thal
- target (heart disease presence)

---

### 🎯 Targets Used in This Experiment

To apply both regression and classification using the same dataset:

- **Linear Regression** → Predict `chol` (cholesterol level – continuous)
- **Logistic Regression** → Predict `target` (0 = no disease, 1 = disease)

Thus, a single Kaggle dataset was used for both models.

---

## 3.Mathematical Formulation

### 🔹 Linear Regression

Linear Regression models:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Objective:

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Where:

- ( y ) = actual cholesterol value
- ( \hat{y} ) = predicted value

---

### 🔹 Logistic Regression

Logistic Regression uses the sigmoid function:

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

Loss Function: Binary Cross Entropy

---

# 4. Algorithm Limitations

### Linear Regression

- Assumes linear relationship
- Sensitive to outliers
- Cannot model non-linear patterns

### Logistic Regression

- Assumes linear decision boundary
- Requires feature scaling
- Performance decreases for highly complex relationships

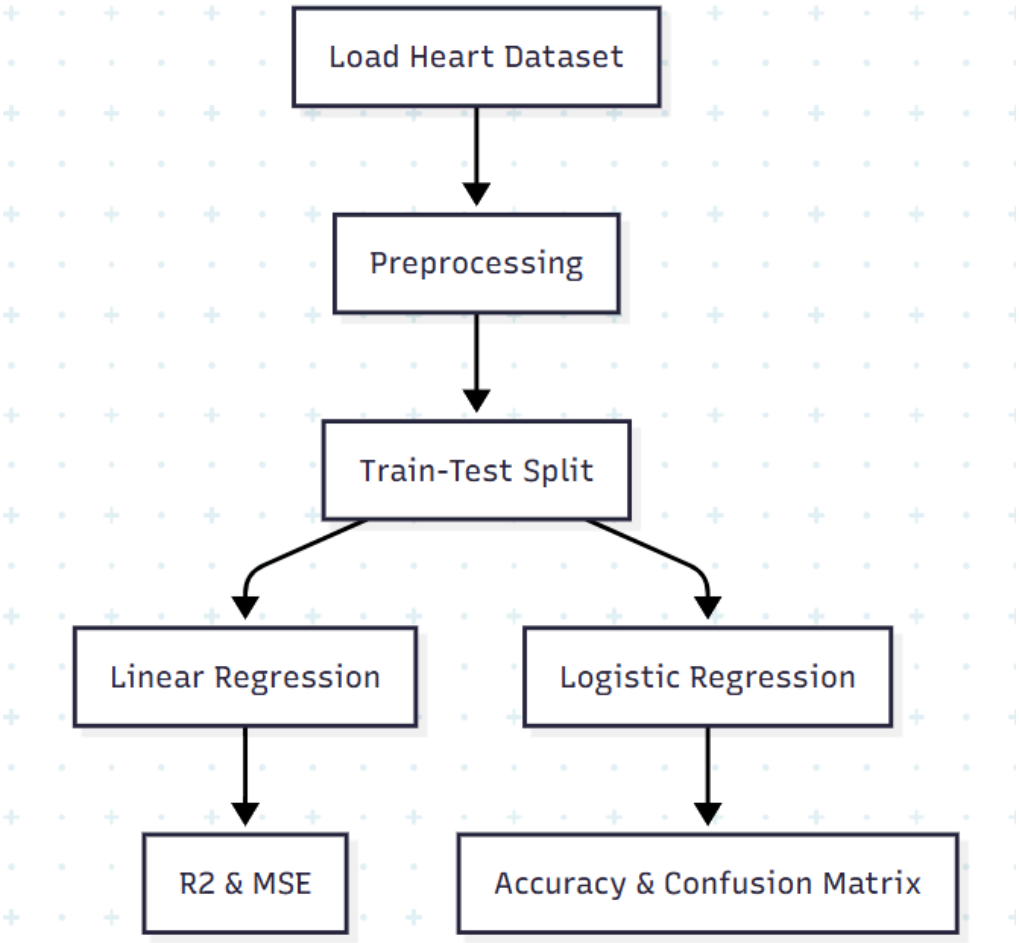
---

# 5. Methodology / Workflow

1. Loaded Heart Disease dataset from Kaggle
2. Checked for missing values
3. Split dataset into training and testing sets
4. Applied Linear Regression to predict cholesterol (`chol`)
5. Evaluated using:
    - $R^2$ Score
    - Mean Squared Error
6. Applied Logistic Regression to predict heart disease (`target`)
7. Scaled features using StandardScaler
8. Evaluated using:
    - Accuracy
    - Confusion Matrix
    - Classification Report

---

### 🔁 Workflow Diagram




---

## 6. Performance Analysis

### 🔹 Linear Regression

- $R^2$ Score indicates how well the model explains variation in cholesterol levels.
- Mean Squared Error measures prediction error.

A moderate $R^2$ indicates the model captures some variation in cholesterol levels.

---

### 🔹 Logistic Regression

- Accuracy Score indicates classification correctness.
- Confusion Matrix shows true positives and true negatives.
- Precision and Recall evaluate classification reliability.

The model achieved high classification accuracy in predicting heart disease presence.

---

# 7.Hyperparameter Tuning

## Linear Regression

- Default parameters used.
- No major hyperparameters.

## Logistic Regression

- `max_iter` increased to ensure convergence.
- Features scaled using StandardScaler.
- Proper train-test split applied to avoid overfitting.

---

# 8.CODE

```
# ==========================================
# STEP 1: Import Libraries
# ==========================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve,
auc, r2_score, mean_squared_error
from sklearn.preprocessing import StandardScaler

# ==========================================
# STEP 2: Load Dataset
# ==========================================
df = pd.read_csv('/content/heart.csv')

X = df.drop("target", axis=1)
y = df["target"]

feature_names = X.columns

# Scaling
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)

# ==========================================
# STEP 3: LINEAR REGRESSION
# ==========================================
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)

y_pred_lin = lin_model.predict(X_test)

print("\n===== LINEAR REGRESSION =====")
print("R2 Score:", r2_score(y_test, y_pred_lin))
print("MSE:", mean_squared_error(y_test, y_pred_lin))

# 🔹 GRAPH: Actual vs Predicted
plt.figure()
plt.scatter(y_test, y_pred_lin)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Linear Regression: Actual vs Predicted")
plt.show()

# ==========================================
# STEP 4: LOGISTIC (BEFORE TUNING)
# ==========================================
model_before = LogisticRegression(max_iter=1000)
model_before.fit(X_train, y_train)

y_pred_before = model_before.predict(X_test)
y_prob_before = model_before.predict_proba(X_test)[:, 1]

acc_before = accuracy_score(y_test, y_pred_before)

print("\n===== LOGISTIC BEFORE =====")
print("Accuracy:", acc_before)
print(classification_report(y_test, y_pred_before))
```

```
# ==========================================
# STEP 5: HYPERPARAMETER TUNING
# ==========================================
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'solver': ['lbfgs', 'liblinear']
}

grid = GridSearchCV(LogisticRegression(max_iter=1000),
                    param_grid,
                    cv=5,
                    scoring='accuracy')

grid.fit(X_train, y_train)

model_after = grid.best_estimator_

# ==========================================
# STEP 6: LOGISTIC (AFTER TUNING)
# ==========================================
y_pred_after = model_after.predict(X_test)
y_prob_after = model_after.predict_proba(X_test)[:, 1]

acc_after = accuracy_score(y_test, y_pred_after)

print("\n===== LOGISTIC AFTER =====")
print("Accuracy:", acc_after)
print(classification_report(y_test, y_pred_after))

# ==========================================
# STEP 7: ACCURACY COMPARISON
# ==========================================
plt.figure()
plt.bar(['Before', 'After'], [acc_before, acc_after])
plt.title("Accuracy Comparison")
plt.show()

# ==========================================
# STEP 8: CONFUSION MATRIX
# ==========================================
fig, axes = plt.subplots(1,2, figsize=(12,5))
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_before), annot=True, fmt='d', ax=axes[0])
axes[0].set_title("Before")

sns.heatmap(confusion_matrix(y_test, y_pred_after), annot=True, fmt='d', ax=axes[1])
axes[1].set_title("After")

plt.show()

# ==========================================
# STEP 9: ROC (SINGLE COMBINED)
# ==========================================
fpr1, tpr1, _ = roc_curve(y_test, y_prob_before)
fpr2, tpr2, _ = roc_curve(y_test, y_prob_after)

plt.figure()
plt.plot(fpr1, tpr1, label="Before")
plt.plot(fpr2, tpr2, label="After")
plt.plot([0,1],[0,1],'--')
plt.title("ROC Curve")
plt.legend()
plt.show()

# ==========================================
# STEP 10: FEATURE IMPORTANCE
# ==========================================
coefficients = model_after.coef_[0]

importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': coefficients
}).sort_values(by='Importance', ascending=False)

plt.figure(figsize=(10,6))
sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title("Feature Importance")
plt.show()
```

# 9.OUTPUT

```
Dataset Shape: (1025, 14)
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
```

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 53 | 1 | 0 | 140 | 203 | 1 | 0 | 155 | 1 | 3.1 | 0 |
| 2 | 70 | 1 | 0 | 145 | 174 | 0 | 1 | 125 | 1 | 2.6 | 0 |
| 3 | 61 | 1 | 0 | 148 | 203 | 0 | 1 | 161 | 0 | 0.0 | 2 |
| 4 | 62 | 0 | 0 | 138 | 294 | 1 | 1 | 106 | 0 | 1.9 | 1 |

| | ca | thal | target |
|---|---|---|---|
| 0 | 2 | 3 | 0 |
| 1 | 0 | 3 | 0 |
| 2 | 0 | 3 | 0 |
| 3 | 1 | 3 | 0 |
| 4 | 3 | 2 | 0 |

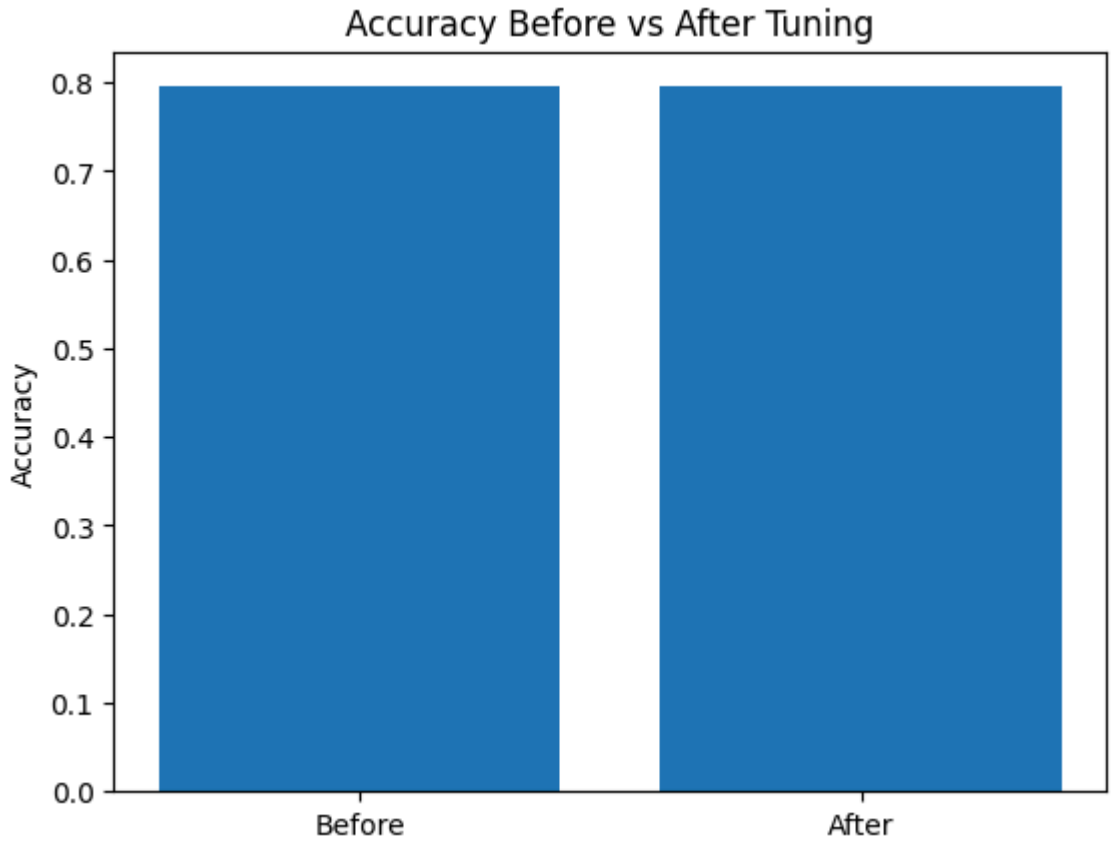
===== BEFORE TUNING =====
Accuracy: 0.7951219512195122

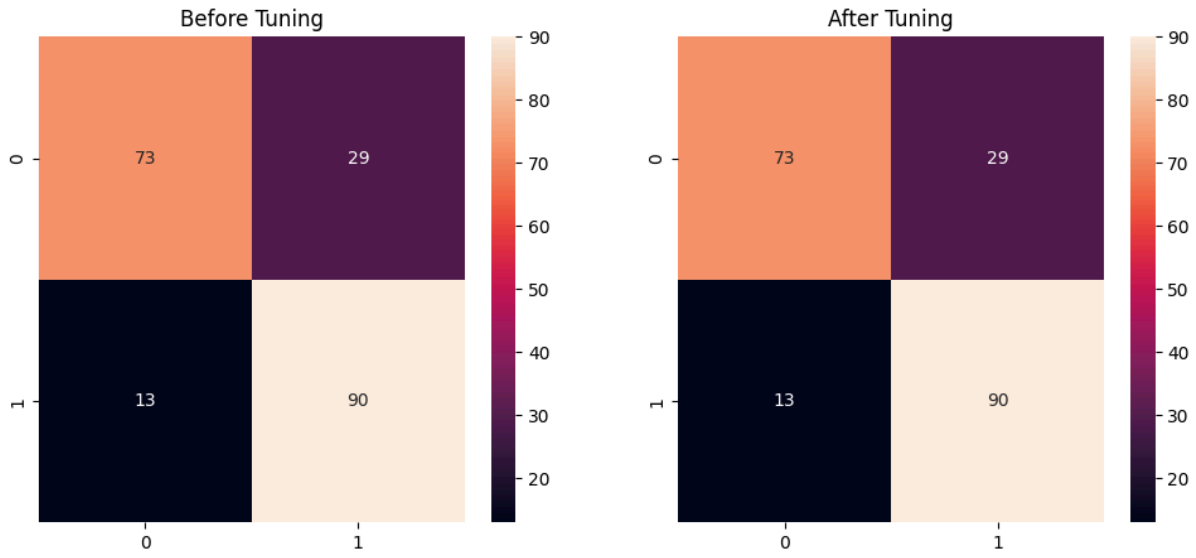
| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.85 | 0.72 | 0.78 | 102 |
| 1 | 0.76 | 0.87 | 0.81 | 103 |
| accuracy | | | 0.80 | 205 |
| macro avg | 0.80 | 0.79 | 0.79 | 205 |
| weighted avg | 0.80 | 0.80 | 0.79 | 205 |

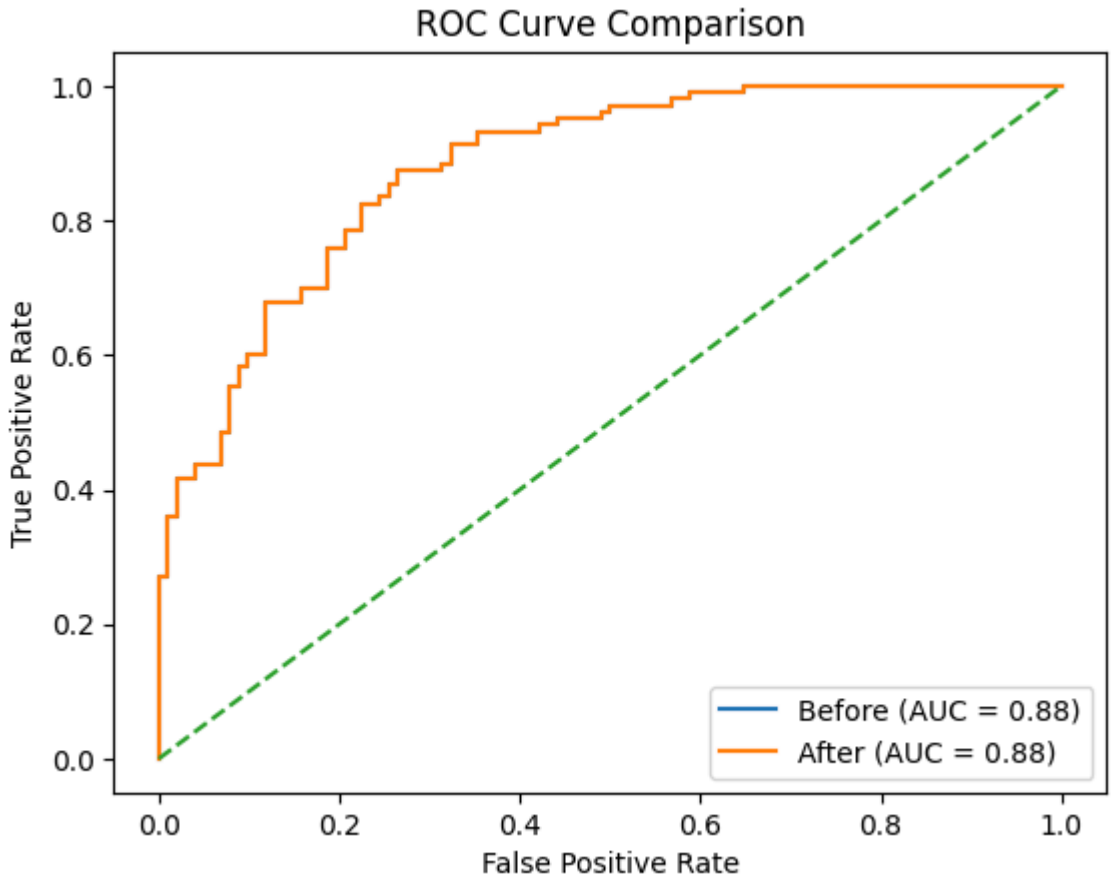
Best Parameters: {'C': 1, 'solver': 'lbfgs'}

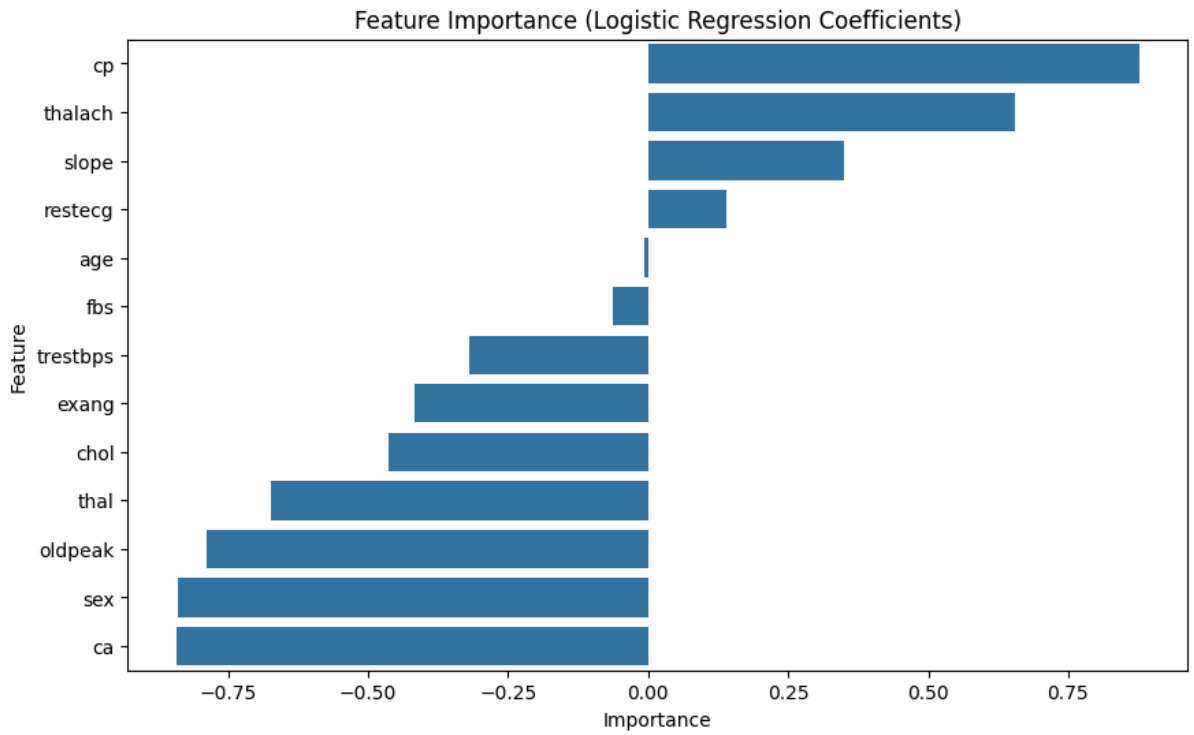
===== AFTER TUNING =====
Accuracy: 0.7951219512195122

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.85 | 0.72 | 0.78 | 102 |
| 1 | 0.76 | 0.87 | 0.81 | 103 |
| accuracy | | | 0.80 | 205 |
| macro avg | 0.80 | 0.79 | 0.79 | 205 |
| weighted avg | 0.80 | 0.80 | 0.79 | 205 |


Accuracy Before vs After Tuning
Accuracy
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
Before
After


Before Tuning
73
29
13
90
0
1
0
1
90
80
70
60
50
40
30
20
After Tuning
73
29
13
90
0
1
0
1
90
80
70
60
50
40
30
20




Top Features:

| | Feature | Importance |
|---|---|---|
| 2 | cp | 0.877341 |

| | | |
|---|---|---|
| 7 | thalach | 0.654871 |
| 10 | slope | 0.349357 |
| 6 | restecg | 0.140684 |
| 0 | age | -0.006754 |

# 10.Conclusion

Linear Regression was successfully used to predict cholesterol levels from medical attributes.

Logistic Regression effectively classified heart disease presence.

Both regression techniques were implemented using the same Kaggle Heart Disease dataset.

The experiment demonstrates practical application of regression models on real-world medical data.

---